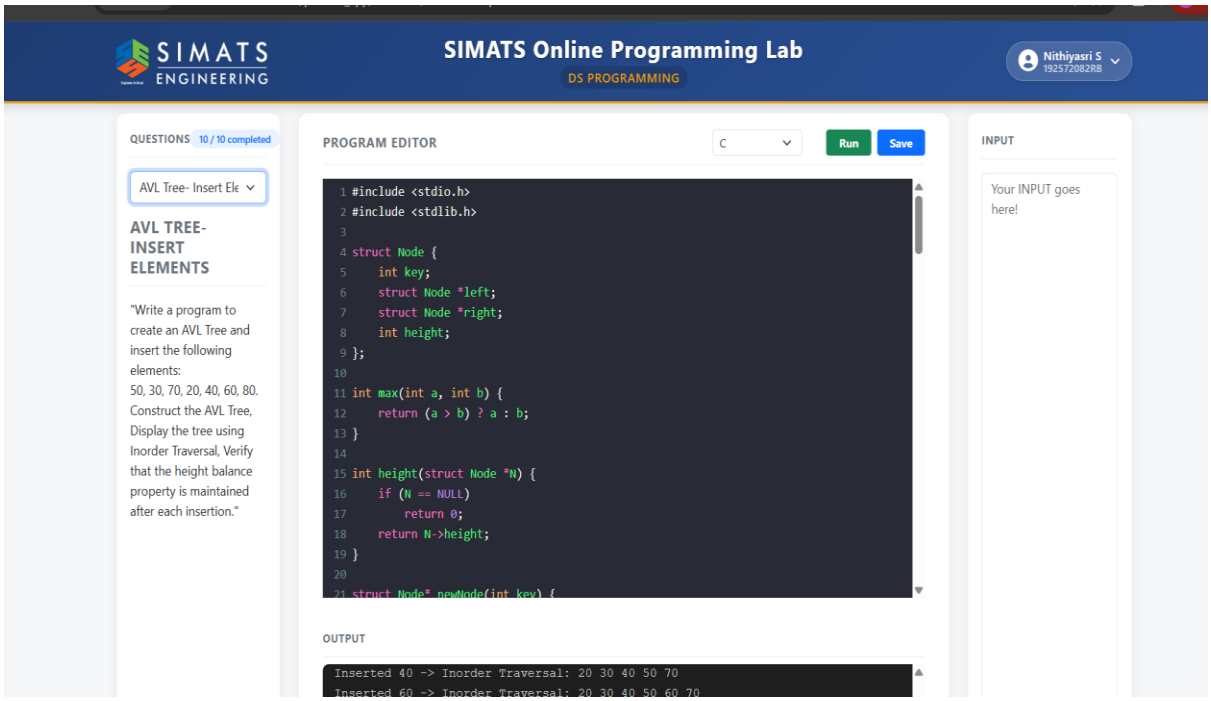
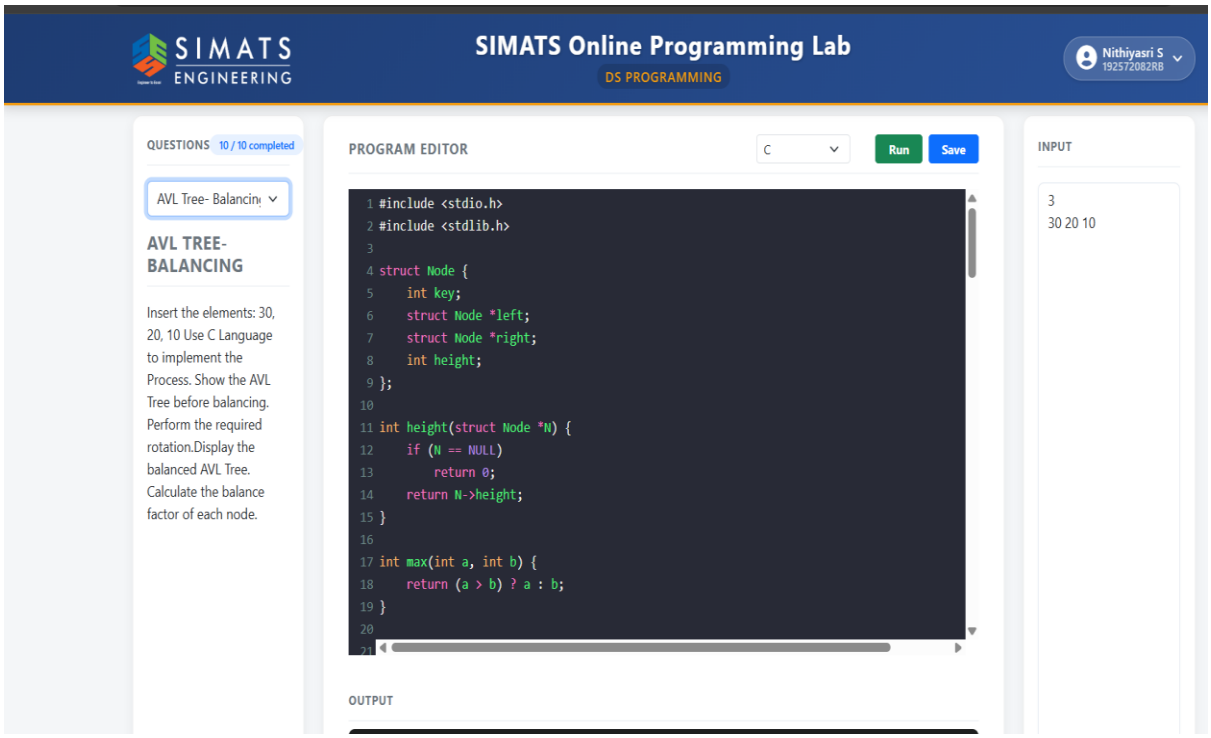
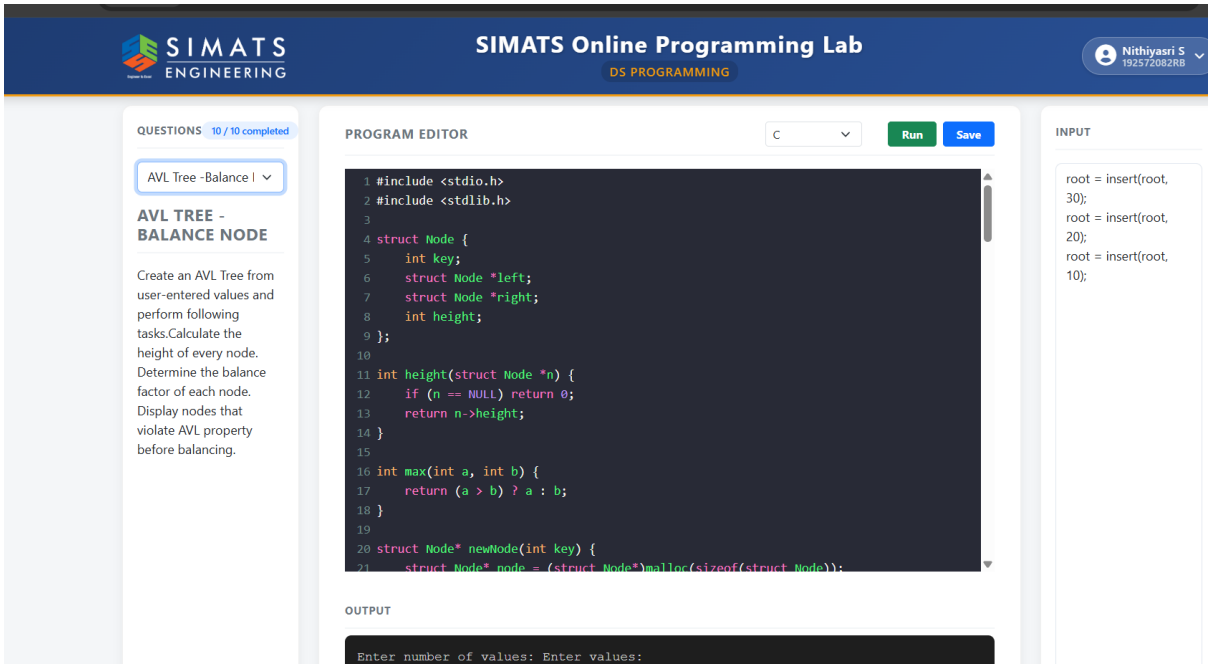
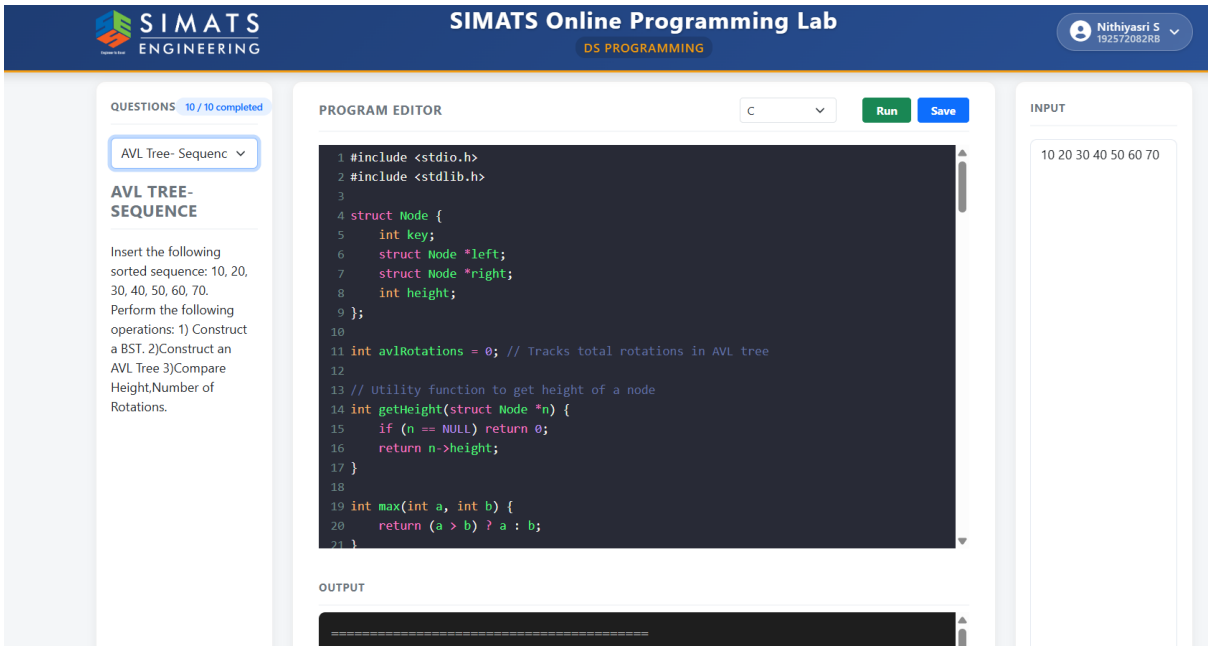
LAB PROGRAMS

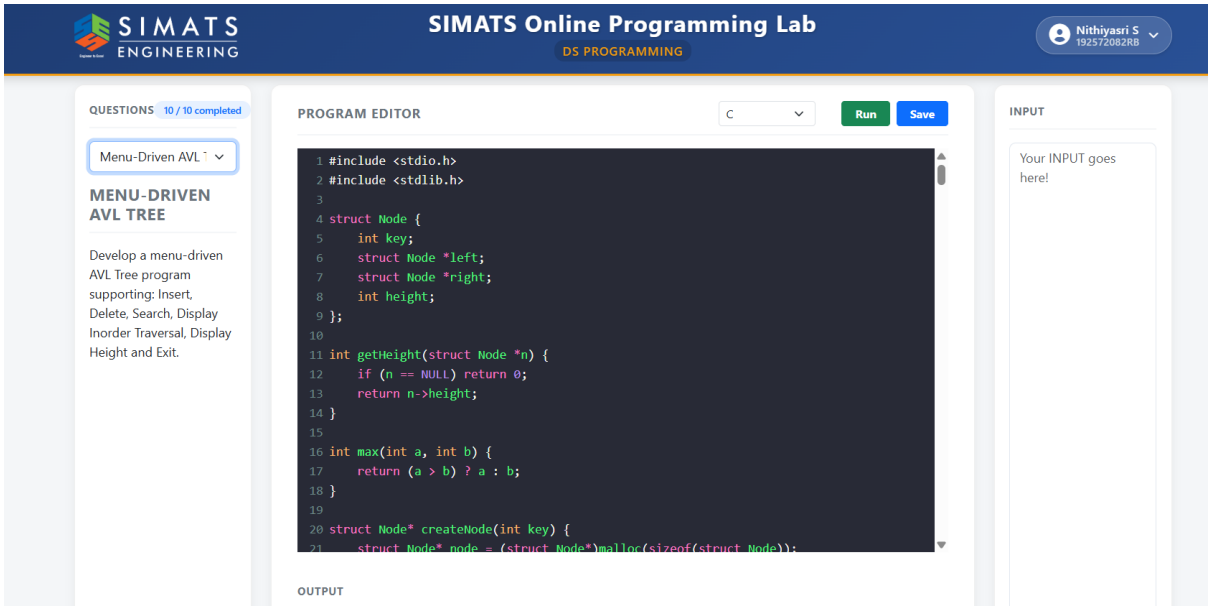
AVL TREE

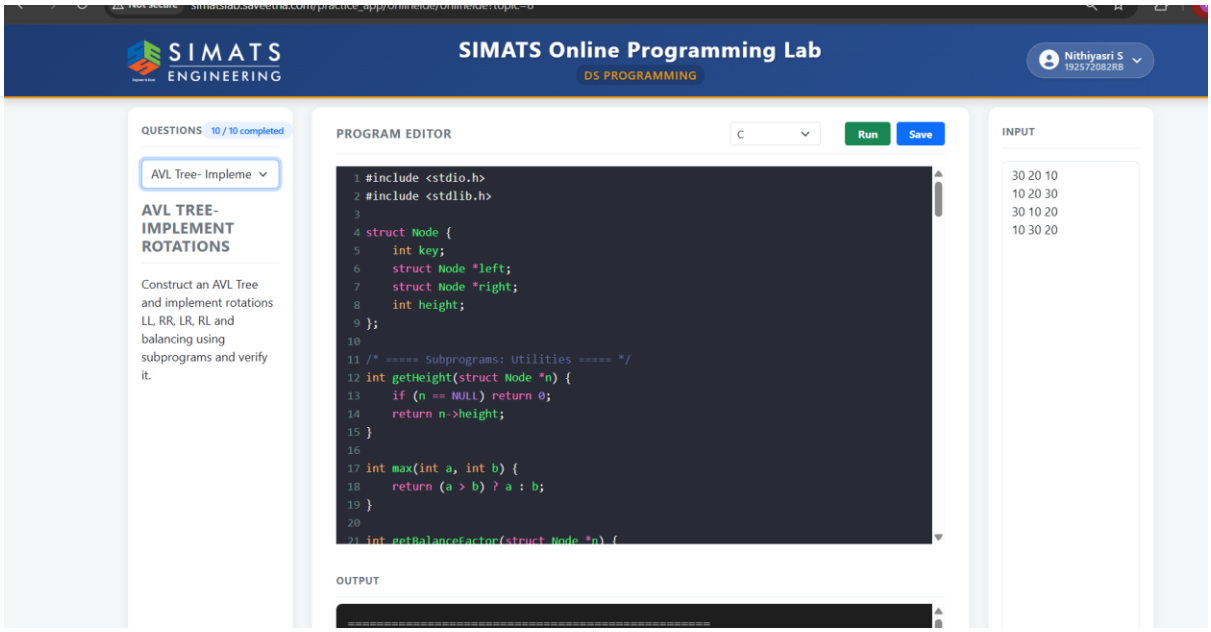


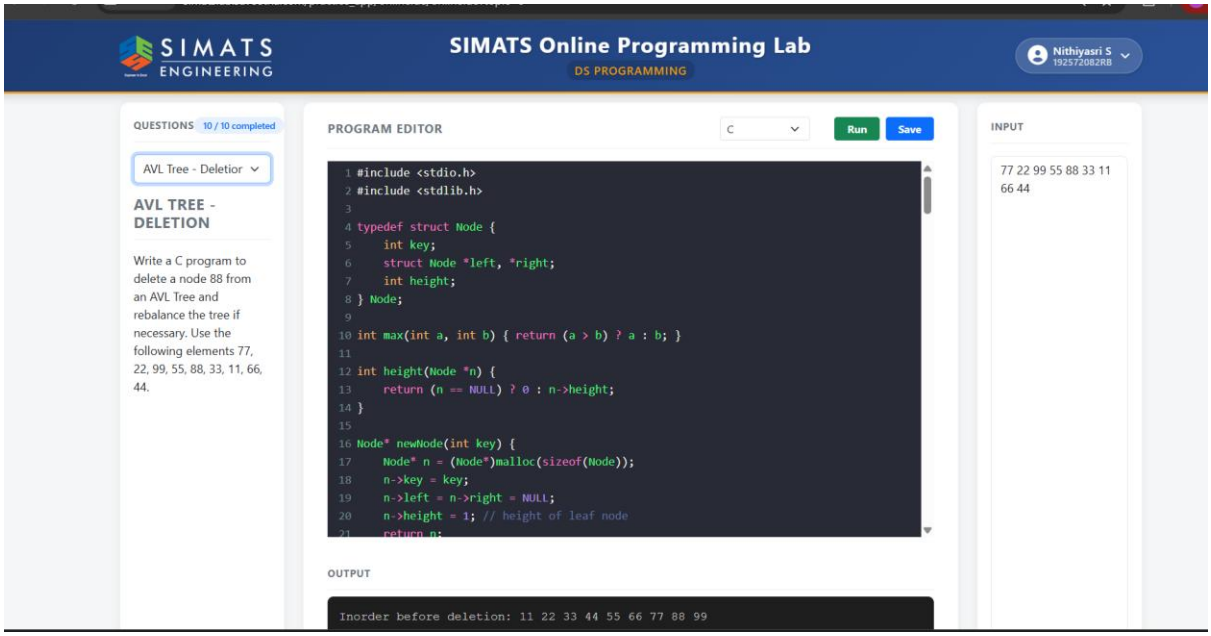


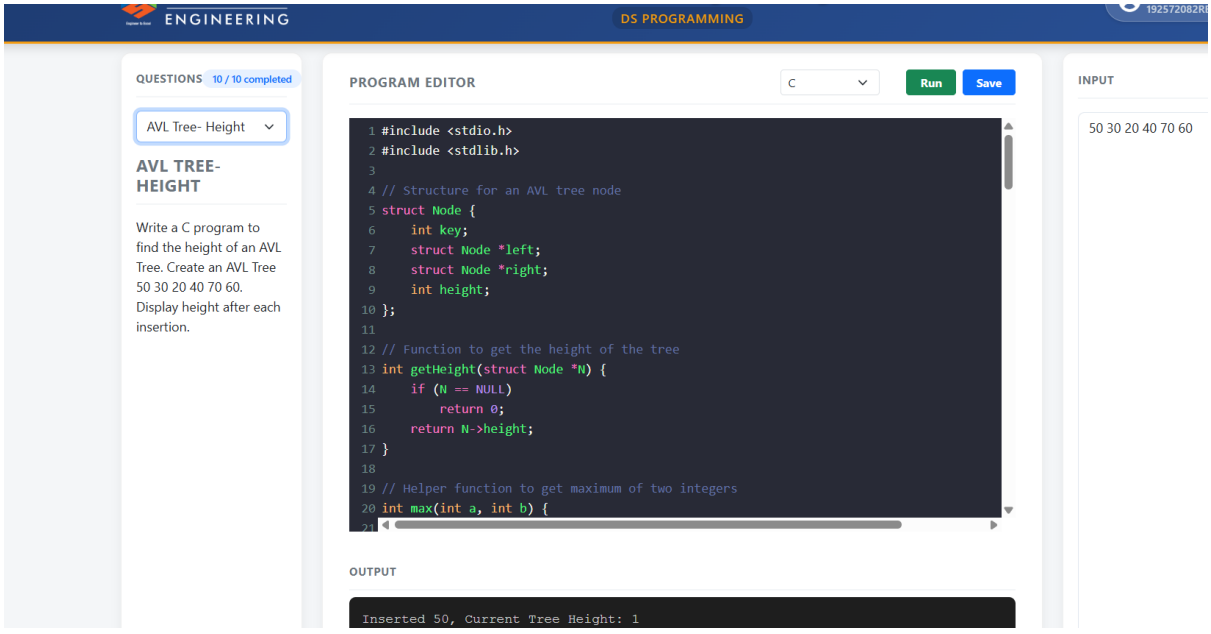

SIMATS ENGINEERING
SIMATS Online Programming Lab
DS PROGRAMMING
Nithiyasri S
192572082RB
QUESTIONS 10 / 10 completed
AVL Tree -Balance I
AVL TREE - BALANCE NODE
Create an AVL Tree from user-entered values and perform following tasks.Calculate the height of every node. Determine the balance factor of each node. Display nodes that violate AVL property before balancing.
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
#include <stdlib.h>
struct Node {
int key;
struct Node *left;
struct Node *right;
int height;
};
int height(struct Node *n) {
if (n == NULL) return 0;
return n->height;
}
int max(int a, int b) {
return (a > b) ? a : b;
}
struct Node* newNode(int key) {
INPUT
root = insert(root, 30);
root = insert(root, 20);
root = insert(root, 10);
OUTPUT
Enter number of values: Enter values:

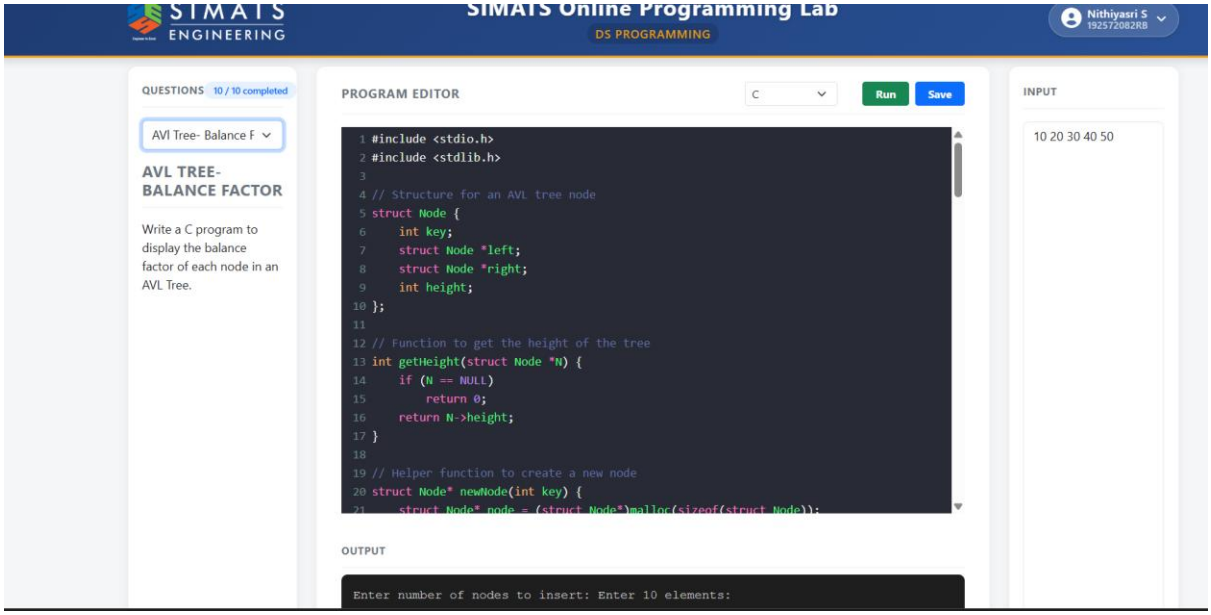

SIMATS ENGINEERING
SIMATS Online Programming Lab
DS PROGRAMMING
Nithiyasri S
192572082RB
QUESTIONS 10 / 10 completed
AVL Tree- Sequenc
AVL TREE- SEQUENCE
Insert the following sorted sequence: 10, 20, 30, 40, 50, 60, 70. Perform the following operations: 1) Construct a BST. 2)Construct an AVL Tree 3)Compare Height,Number of Rotations.
PROGRAM EDITOR
C
Run
Save
#include <stdio.h>
#include <stdlib.h>
struct Node {
int key;
struct Node *left;
struct Node *right;
int height;
};
int avlRotations = 0; // Tracks total rotations in AVL tree
// Utility function to get height of a node
int getHeight(struct Node *n) {
if (n == NULL) return 0;
return n->height;
}
int max(int a, int b) {
return (a > b) ? a : b;
INPUT
10 20 30 40 50 60 70
OUTPUT

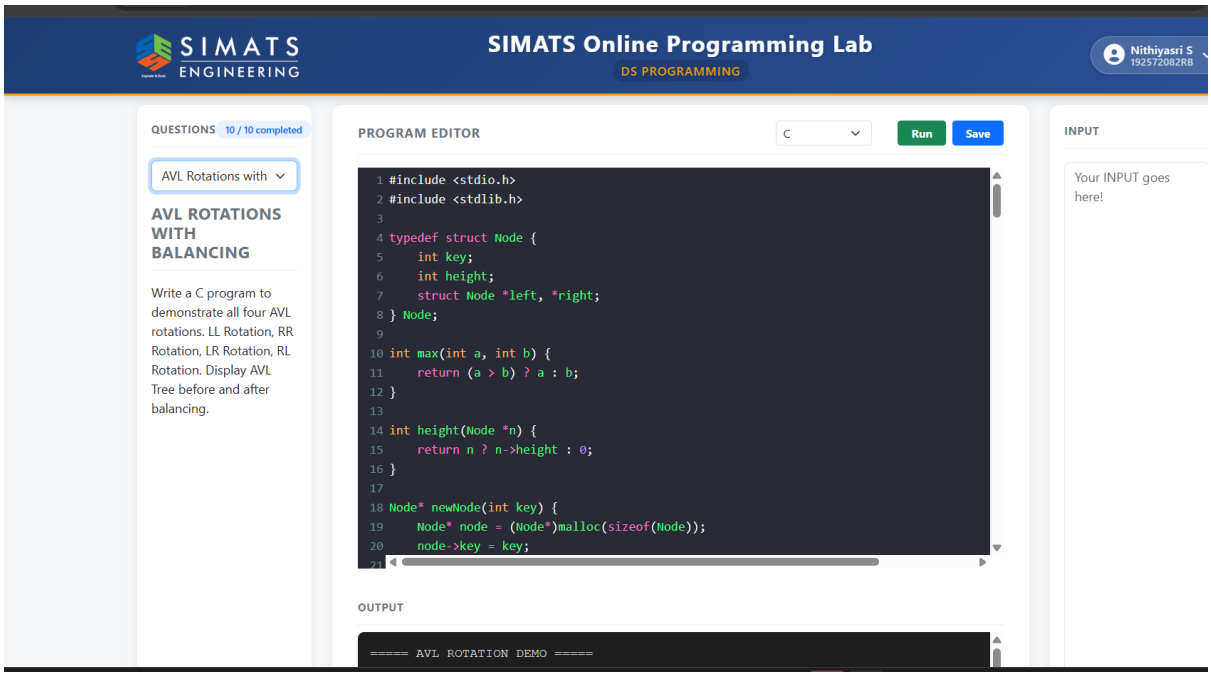

SIMATS ENGINEERING
SIMATS Online Programming Lab
DS PROGRAMMING
Nithiyasri S
192572082RB
QUESTIONS 10 / 10 completed
Menu-Driven AVL
MENU-DRIVEN AVL TREE
Develop a menu-driven AVL Tree program supporting: Insert, Delete, Search, Display Inorder Traversal, Display Height and Exit.
PROGRAM EDITOR
C
Run
Save
INPUT
Your INPUT goes here!
#include <stdio.h>
#include <stdlib.h>
struct Node {
int key;
struct Node *left;
struct Node *right;
int height;
};
int getHeight(struct Node *n) {
if (n == NULL) return 0;
return n->height;
}
int max(int a, int b) {
return (a > b) ? a : b;
}
struct Node* createNode(int key) {
OUTPUT


SIMATS ENGINEERING
SIMATS Online Programming Lab
DS PROGRAMMING
Nithiyasri S
192572082RB
QUESTIONS 10 / 10 completed
AVL Tree- Impleme
AVL TREE-IMPLEMENT ROTATIONS
Construct an AVL Tree and implement rotations LL, RR, LR, RL and balancing using subprograms and verify it.
PROGRAM EDITOR
C
Run
Save
INPUT
30 20 10
10 20 30
30 10 20
10 30 20
#include <stdio.h>
#include <stdlib.h>
struct Node {
int key;
struct Node *left;
struct Node *right;
int height;
};
/* ===== Subprograms: Utilities ===== */
int getHeight(struct Node *n) {
if (n == NULL) return 0;
return n->height;
}
int max(int a, int b) {
return (a > b) ? a : b;
}
OUTPUT


SIMATS ENGINEERING
SIMATS Online Programming Lab
DS PROGRAMMING
Nithiyasri S
192572082RB
QUESTIONS 10 / 10 completed
AVL Tree - Deletior
AVL TREE - DELETION
Write a C program to delete a node 88 from an AVL Tree and rebalance the tree if necessary. Use the following elements 77, 22, 99, 55, 88, 33, 11, 66, 44.
PROGRAM EDITOR
C
Run
Save
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5 int key;
6 struct Node *left, *right;
7 int height;
8 } Node;
9
10 int max(int a, int b) { return (a > b) ? a : b; }
11
12 int height(Node *n) {
13 return (n == NULL) ? 0 : n->height;
14 }
15
16 Node* newNode(int key) {
17 Node* n = (Node*)malloc(sizeof(Node));
18 n->key = key;
19 n->left = n->right = NULL;
20 n->height = 1; // height of leaf node
21 return n;
INPUT
77 22 99 55 88 33 11 66 44
OUTPUT
Inorder before deletion: 11 22 33 44 55 66 77 88 99


ENGINEERING
DS PROGRAMMING
QUESTIONS 10 / 10 completed
AVL Tree- Height
AVL TREE-HEIGHT
Write a C program to find the height of an AVL Tree. Create an AVL Tree 50 30 20 40 70 60. Display height after each insertion.
PROGRAM EDITOR
C
Run
Save
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Structure for an AVL tree node
5 struct Node {
6 int key;
7 struct Node *left;
8 struct Node *right;
9 int height;
10 };
11
12 // Function to get the height of the tree
13 int getHeight(struct Node *N) {
14 if (N == NULL)
15 return 0;
16 return N->height;
17 }
18
19 // Helper function to get maximum of two integers
20 int max(int a, int b) {
INPUT
50 30 20 40 70 60
OUTPUT
Inserted 50, Current Tree Height: 1


SIMATS ENGINEERING
SIMATS Online Programming Lab
DS PROGRAMMING
Nithiyasri S
192572082RB
QUESTIONS 10 / 10 completed
AVl Tree- Balance F
AVL TREE-BALANCE FACTOR
Write a C program to display the balance factor of each node in an AVL Tree.
PROGRAM EDITOR
C
Run
Save
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Structure for an AVL tree node
5 struct Node {
6 int key;
7 struct Node *left;
8 struct Node *right;
9 int height;
10 };
11
12 // Function to get the height of the tree
13 int getHeight(struct Node *N) {
14 if (N == NULL)
15 return 0;
16 return N->height;
17 }
18
19 // Helper function to create a new node
20 struct Node* newNode(int key) {
21 struct Node* node = (struct Node*)malloc(sizeof(struct Node));
INPUT
10 20 30 40 50
OUTPUT
Enter number of nodes to insert: Enter 10 elements:


SIMATS ENGINEERING
SIMATS Online Programming Lab
DS PROGRAMMING
Nithiyasri S
192572082RB
QUESTIONS 10 / 10 completed
AVL Rotations with
AVL ROTATIONS WITH BALANCING
Write a C program to demonstrate all four AVL rotations. LL Rotation, RR Rotation, LR Rotation, RL Rotation. Display AVL Tree before and after balancing.
PROGRAM EDITOR
C
Run
Save
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5 int key;
6 int height;
7 struct Node *left, *right;
8 } Node;
9
10 int max(int a, int b) {
11 return (a > b) ? a : b;
12 }
13
14 int height(Node *n) {
15 return n ? n->height : 0;
16 }
17
18 Node* newNode(int key) {
19 Node* node = (Node*)malloc(sizeof(Node));
20 node->key = key;
21
INPUT
Your INPUT goes here!
OUTPUT
===== AVL ROTATION DEMO =====